

**frogger.zip**
File

I am working on Lab 4: Vibe Coding for my PES University CSE course.

I have attached the complete starter project for the assigned Pygame Frogger application as frogger.zip.

Repository:

https://github.com/anisha12git/01_frogger

I have already:

1. Forked the assigned repository to my personal GitHub account.
2. Cloned my fork locally.
3. Installed the required Python dependencies.
4. Run the original game successfully.
5. Recorded the required 10-second video showing the original broken behaviour.

According to the project README, the first task is:

Task 1 — Fix Vehicle Collision Detection

"Fix the existing vehicle collision behaviour so that collisions are detected correctly."

I tested the original game and observed that when the frog comes into contact with a vehicle, the current behaviour is incorrect.

Please inspect all of the attached project files before proposing a solution.

For this prompt, work ONLY on Task 1.

Please:

1. Identify the existing vehicle collision detection implementation.
2. Determine the specific cause of the incorrect collision behaviour.
3. Explain the cause clearly before making changes.
4. Identify exactly which file(s) need to be modified.
5. Provide the minimal code changes required to fix the collision

detection.

6. Preserve the existing project structure, controls, rendering, and unrelated functionality.
7. Do not implement the lives/respawn system yet.
8. Do not implement goal detection or score tracking yet.
9. Do not implement the timer yet.
10. Provide the complete updated contents of any modified file so that I can apply the changes accurately.
11. Provide a short testing procedure that I can use to verify that Task 1 works correctly.

Please base the solution on the actual uploaded source code rather than making assumptions about the implementation.

Worked for 13s

I inspected the actual contents of `frogger.zip`, including the game logic files, rather than assuming how the starter project works.

1. Existing vehicle collision implementation

The collision logic is in:

`frogger/game/collisions.py`

The current implementation is:

`</>` Python

```
def check_collision(frog, vehicles):  
    """  
    Returns True if the frog is currently hit by any vehicle.  
    """  
    for v in vehicles:  
        vehicle_col = int(v.x // CELL_SIZE)  
        if vehicle_col == frog.col and v.row == frog.row:  
            return True  
    return False
```

So the game currently decides that a collision happened when:

1. The vehicle's **left x-coordinate** falls inside the frog's grid column, and
2. The vehicle is in the same row as the frog.

2. The specific bug

The problem is that **the vehicle is not grid-sized like the frog**.

The frog occupies one 50×50 grid cell, but vehicles move continuously in pixel space.

From `vehicle.py` :

`</>` Python

```
self.x = float(x)
self.width = width
```

and vehicles have two different widths in `game_engine.py` :

`</>` Python

```
vehicle_width = 40 if i % 2 == 0 else 70
```

So some vehicles are **70 pixels wide**, which is larger than the 50-pixel grid cell.

The current collision code only checks:

`</>` Python

```
int(v.x // CELL_SIZE) == frog.col
```

That checks **where the vehicle starts**, not whether the actual vehicle body overlaps the frog.

Example

Suppose:

- Frog occupies column 5 → x = 250–300
- A 70-pixel vehicle starts at x = 290

The vehicle occupies:

`</>` Plain text

```
290 ----- 360
      overlaps
250 ----- 300
      FROG
```

There is a real collision from x=290 to x=300.

But:

</> Python

```
290 // 50 = 5
```

so this particular case happens to work.

Now consider a vehicle starting at x=240:

</> Plain text

```
240 ----- 310
      overlaps
250 ----- 300
      FROG
```

There is clearly a collision.

But:

</> Python

```
240 // 50 = 4
```

while the frog is in column 5.

Therefore the existing code returns **False even though the vehicle overlaps the frog.**

The fundamental issue is:

Collision is being approximated using grid-column equality instead of checking the actual rectangular boundaries of the moving vehicle and frog.

This is especially incorrect because the vehicle moves continuously rather than snapping to the grid.

3. The correct fix

Fortunately, the project already provides exactly what we need.

Frog has:

</> Python

```
def get_rect(self, cell_size, pad=4):
```

and `Vehicle` has:

`</>` Python

```
def get_rect(self, cell_size, pad=4):
```

Both return `pygame.Rect` objects.

Therefore, Task 1 should use Pygame's rectangle collision detection:

`</>` Python

```
frog_rect.collidect(vehicle_rect)
```

This checks actual geometric overlap between the frog and vehicle.

4. File that needs modification

Only **one file** needs to be changed:

`</>` Plain text

```
frogger/game/collisions.py
```

No changes are required to:

- `frogger/game/frog.py`
- `frogger/game/vehicle.py`
- `frogger/game/game_engine.py`
- `frogger/game/renderer.py`
- `frogger/main.py`

This preserves the existing project structure and all unrelated functionality.

The existing `GameEngine` already does:

`</>` Python

```
if check_collision(self.frog, self.vehicles):  
    self.frog.reset()
```

So once `check_collision()` correctly detects the collision, the existing reset behaviour will work automatically.

5. Complete updated `collisions.py`

Replace the contents of `frogger/game/collisions.py` with:

`</>` Python

```
"""
collisions: frog-vs-vehicle collision detection.
"""

CELL_SIZE = 50

def check_collision(frog, vehicles):
    """
    Returns True if the frog is currently hit by any vehicle.
    """
    frog_rect = frog.get_rect(CELL_SIZE)

    for v in vehicles:
        vehicle_rect = v.get_rect(CELL_SIZE)

        if frog_rect.colliderect(vehicle_rect):
            return True

    return False
```

What changed?

The old logic:

`</>` Python

```
vehicle_col = int(v.x // CELL_SIZE)
if vehicle_col == frog.col and v.row == frog.row:
```

has been replaced with:

`</>` Python

```
frog_rect = frog.get_rect(CELL_SIZE)

for v in vehicles:
```

```
vehicle_rect = v.get_rect(CELL_SIZE)

if frog_rect.colliderect(vehicle_rect):
    return True
```

That's the **minimal functional change** required for Task 1.

6. Why this preserves the existing game

Nothing else changes.

The frog still:

- moves one grid cell at a time
- uses the same arrow-key controls
- has the same rendering
- resets when hit

Vehicles still:

- move continuously
- have the same speeds
- have the same widths
- wrap around the screen

The goal behaviour remains untouched:

 Python

```
if self.frog.row == GOAL_ROW:
    self.frog.reset()
```

And importantly, we do not add anything for:

- lives
- respawning
- score
- timer
- goal tracking

Those are later tasks.

7. Testing procedure for Task 1

After replacing `collisions.py` :

Test 1 — Vehicle clearly misses frog

1. Run:

```
</> Bash
```

```
python main.py
```

2. Move the frog onto a road lane.
3. Move/stay in a position where the vehicle passes nearby but does **not** touch the frog.
4. The frog should **not** reset.

Test 2 — Vehicle hits frog

1. Move the frog onto a road lane.
2. Allow a vehicle to move toward the frog.
3. When the vehicle visibly touches/overlaps the frog, the frog should immediately reset to the starting position.

Test 3 — Partial overlap

This is the important test for the original bug.

Position the frog so that a vehicle approaches from the side and only **part of the vehicle overlaps the frog**.

The frog should reset as soon as their rectangles overlap.

This verifies that collision is based on **actual geometry**, rather than whether the vehicle's starting x-coordinate happens to fall inside the frog's grid column.

Test 4 — Different vehicle sizes

The project deliberately uses:

```
</> Python
```

```
40px
```

vehicles and:

```
</> Python
```

```
70px
```


vehicles.

Test both types. Both should trigger collision correctly when they physically overlap the frog.

Final Task 1 change

Modified file: `frogger/game/collisions.py`

Root cause: Collision was being detected by comparing the vehicle's starting grid column with the frog's grid column, which is incorrect for continuously moving vehicles, especially the 70-pixel-wide vehicles.

Fix: Use the already-existing `Frog.get_rect()` and `Vehicle.get_rect()` methods with Pygame's `Rect.collidect()` .

This is the smallest change that correctly fixes **Task 1 only**.

Task 1 has been implemented and tested successfully.

Please now work **ONLY** on Task 2 from the project README.

Task 2 — Implement Lives and Respawn

"Add a 3-life system with appropriate frog respawn behaviour after a collision."

Requirements:

- The player should begin with 3 lives.
- A vehicle collision should cause exactly one life to be lost.
- After losing a life, the frog should respawn at the appropriate starting position.
- The game should continue if the player still has remaining lives.
- Losing all 3 lives should result in a clear Game Over state.
- Pressing R should restart the game and restore the initial 3 lives.
- The number of remaining lives should be displayed during gameplay.
- The vehicle collision detection fix from Task 1 must remain functional.
- Existing controls and game behaviour should be preserved.

Please inspect the current project files and implement this functionality using the existing project structure.

For this task:

1. Explain the implementation approach before providing the changes.
2. Identify the exact file(s) that need to be modified.
3. Make only the changes necessary for Task 2.
4. Preserve the implementation from Task 1.
5. Do not implement Task 3 (goal and score tracking).
6. Do not implement Task 4 (30-second timer).
7. Do not rewrite unrelated parts of the project.
8. Provide the complete updated contents of any modified file.
9. Provide a concise testing procedure covering:
 - Starting with 3 lives
 - Losing one life through a collision
 - Frog respawning
 - Continuing with remaining lives
 - Game Over after all lives are lost
 - Restarting with R

Worked for 26s

I inspected the uploaded project and implemented **Task 2 only**.

One important note: the uploaded ZIP still contains the **pre-Task-1 version** of `collisions.py`. Since you said Task 1 is already completed, I have carried the correct Task 1 rectangle-collision implementation forward in that file so the Task 2 changes do not regress it.

Implementation approach

The lives system is kept inside `GameEngine`, because it already owns the frog, vehicles, collision handling, restart behaviour, and per-frame game state.

The implementation works as follows:

- `INITIAL_LIVES = 3`
- `self.lives` stores the current number of lives.
- `self.game_over` tracks whether all lives have been exhausted.
- On a collision:
 - exactly **one** life is deducted;
 - if lives remain, `frog.reset()` respawns the frog;
 - if lives reach `0`, `game_over` becomes `True`.
- While Game Over:
 - vehicle updates stop;

- frog movement is disabled;
 - the Game Over message remains visible.
- R always:
 - restores 3 lives;
 - clears Game Over;
 - rebuilds the game entities;
 - places the frog back at its starting position.
- The renderer displays Lives: X .
- The existing Task 1 rectangle collision detection remains intact.
- Existing goal-reset behaviour remains unchanged; **no score/goal system has been added.**

Files modified

Only these three files need changes:

1. frogger/game/game_engine.py
2. frogger/game/collisions.py — retains the Task 1 fix
3. frogger/game/renderer.py — displays lives and Game Over

frog.py , vehicle.py , and main.py do **not** need modification.

1. frogger/game/collisions.py

This retains the Task 1 fix using actual rectangle intersection.

 Python

```

"""
collisions: frog-vs-vehicle collision detection.
"""

CELL_SIZE = 50

def check_collision(frog, vehicles):
    """
    Returns True if the frog is currently hit by any vehicle.
    """
    frog_rect = frog.get_rect(CELL_SIZE)

    for v in vehicles:
        vehicle_rect = v.get_rect(CELL_SIZE)
        if frog_rect.colliderect(vehicle_rect):
            return True

    return False

```

2. frogger/game/game_engine.py

This is where the actual Task 2 functionality is implemented.

</> Python

```
"""
```

```
GameEngine: owns the frog and all vehicles, and runs one frame's worth
of game logic.
```

```
Task 2 adds a three-life system, frog respawn behaviour, and a Game Over
state. Goal, score, and timer functionality are intentionally unchanged.
```

```
"""
```

```
import random
```

```
import pygame
```

```
from game.frog import Frog
```

```
from game.vehicle import Vehicle
```

```
from game.collisions import check_collision
```

```
from game.renderer import (
```

```
    GRID_COLS, GRID_ROWS, GOAL_ROW, ROAD_ROWS, START_ROW, CELL_SIZE, WIDTH, HEIGHT,
```

```
)
```

```
LANE_SPEEDS = [1.5, -2, 2, -2.5, 1.5, -2] # one entry per road row, alternating direction
```

Important part

The collision handling is:

</> Python

```
if check_collision(self.frog, self.vehicles):
```

```
    self.lives -= 1
```

```
    if self.lives <= 0:
```

```
        self.lives = 0
```

```
        self.game_over = True
```

```
    else:
```

```
        self.frog.reset()
```

So one collision causes **one** life loss and one respawn.

Once `game_over` is reached, `update()` immediately returns, preventing additional life deductions.

3. frogger/game/renderer.py

The only Task 2 addition here is the ability to display the lives counter and the Game Over banner.

</> Python

```
"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame

CELL_SIZE = 50
GRID_COLS = 12
GRID_ROWS = 8

WIDTH = CELL_SIZE * GRID_COLS
HEIGHT = CELL_SIZE * GRID_ROWS
WINDOW_SIZE = (WIDTH, HEIGHT)

GOAL_ROW = 0
ROAD_ROWS = list(range(1, GRID_ROWS - 1)) # rows 1..6
START_ROW = GRID_ROWS - 1                 # row 7

COLOR_BG = (20, 20, 25)
COLOR_GOAL = (40, 130, 60)
```

I also verified that the modified Python files compile successfully.

Testing procedure

Run the game normally:

</> Bash

```
python main.py
```

1. Starting with 3 lives

Immediately after launching:

</> Plain text

```
Lives: 3
```

should appear on screen.

2. Lose one life

Move the frog onto a road lane and allow a vehicle to collide with it.

Expected:

</> Plain text

Lives: 2

The frog should immediately return to its starting position.

3. Continue playing

Move the frog again after the collision.

Expected:

- Arrow keys still work.
- Vehicles continue moving.
- The game does **not** end after the first collision.
- Another collision should reduce lives from 2 → 1.

4. Lose the third life

Cause another collision.

Expected:

</> Plain text

Lives: 0

and:

</> Plain text

GAME OVER - Press R to restart

The game should no longer respond to arrow-key movement.

5. Restart with R

Press:

</> Plain text

R

Expected:

- Lives return to 3.
- Game Over disappears.
- Frog returns to its starting position.
- Vehicles are rebuilt.
- Arrow-key movement works again.

6. Verify Task 1 wasn't broken

During testing, make sure a vehicle that **partially overlaps the frog** still causes a collision. This confirms the rectangle-based Task 1 fix is still being used.

No Task 3 score/goal tracking or Task 4 timer functionality has been added.

Tasks 1 and 2 have been implemented and tested successfully.

Please now work **ONLY** on Task 3 from the project README.

Task 3 — Implement Goal and Score Tracking

"Detect successful goal completion and implement score tracking with an appropriate win state."

Requirements:

- Detect when the frog successfully reaches the existing goal zone.
- Reaching the goal should result in a clear Win state.
- The current score should be maintained and displayed during gameplay.
- The implementation should use the existing goal zone and project structure.
- The scoring mechanism should be simple and appropriate for the existing game.
- The collision detection and 3-life/respawn functionality from Tasks 1 and 2 must continue to work correctly.
- Pressing R should still restart the game correctly.
- Existing controls and unrelated functionality should be preserved.

Please inspect the current project implementation before making changes.

For this task:

1. Explain how goal detection will be implemented.
2. Explain how score tracking will be implemented.
3. Explain how the Win state will be represented.
4. Identify the exact file(s) that need to be modified.
5. Provide the minimal required code changes.
6. Preserve all functionality from Tasks 1 and 2.
7. Do not implement the 30-second timer yet.
8. Do not modify unrelated code.
9. Provide the complete updated contents of any modified file.
10. Provide a concise testing procedure covering:
 - Reaching the goal
 - Win state
 - Score display
 - Vehicle collision
 - Lives and respawn
 - Game Over
 - R restart

Worked for 19s

Yes. I inspected the uploaded project before making the Task 3 plan.

One important thing I found: the ZIP you uploaded is still the original starter repository; it does **not** contain the Task 1 and Task 2 changes you said you already implemented locally. So I cannot honestly claim those changes are present in the uploaded files.

I'll therefore preserve the Task 1 + Task 2 implementation we established previously and add **only** Task 3 on top of it.

1. Goal detection

The project already defines:

</> Python

```
GOAL_ROW = 0
```

and the frog moves using discrete grid rows.

So Task 3 does not need a new goal system. We simply detect:

</> Python


```
self.frog.row == GOAL_ROW
```

When that happens:

- the player has successfully reached the existing green goal zone;
- `self.won` becomes `True` ;
- the game enters a Win state;
- the frog is **not immediately reset**, so the successful position remains visible.

Once `won` is `True` , movement and vehicle updates stop.

2. Score tracking

For the simple scoring mechanism appropriate to this project:

- The game starts at **0 points**.
- Successfully reaching the goal awards **100 points**.
- The score is displayed continuously during gameplay.
- The score is preserved while the player is playing.
- Pressing `R` resets the score to `0` .

The score is awarded only once because the game enters the Win state immediately after reaching the goal.

3. Win state

The `GameEngine` gets:

 Python

```
self.won = False
```

When the frog reaches `GOAL_ROW` :

 Python

```
self.score += 100  
self.won = True
```

While either:

 Python

```
self.game_over
```

or:

```
</> Python
```

```
self.won
```

is true, normal gameplay stops.

The renderer displays:

```
</> Plain text
```

```
YOU WIN! Score: 100
```

This is deliberately separate from the future timer functionality.

4. Files that need modification

Only **two files** need to be changed for Task 3:

1. frogger/game/game_engine.py
2. frogger/game/renderer.py

collisions.py remains unchanged from Task 1.

frog.py , vehicle.py , and main.py remain unchanged.

5. Complete updated game_engine.py

This version contains the Task 1 collision fix, Task 2 lives/respawn system, and Task 3 goal/score/win functionality.

```
</> Python
```

```
"""
```

```
GameEngine: owns the frog and all vehicles, and runs one frame's worth
of game logic.
```

```
Task 2:
```

- 3-life system
- frog respawn after collision

- Game Over state
- R restart

Task 3:

- goal detection
- score tracking
- Win state

The timer is intentionally not implemented yet.

"""

import random

6. Complete updated renderer.py

There is actually very little Task 3-specific rendering required because the existing renderer already has `draw_text()` and `draw_banner()` .

The updated file is:

`</>` Python

"""

renderer: all pygame drawing lives here, kept separate from game logic.

"""

import pygame

CELL_SIZE = 50

GRID_COLS = 12

GRID_ROWS = 8

WIDTH = CELL_SIZE * GRID_COLS

HEIGHT = CELL_SIZE * GRID_ROWS

WINDOW_SIZE = (WIDTH, HEIGHT)

GOAL_ROW = 0

ROAD_ROWS = list(range(1, GRID_ROWS - 1))

START_ROW = GRID_ROWS - 1

COLOR_BG = (20, 20, 25)

COLOR_GOAL = (40, 130, 60)

7. Task 3 testing

Test 1 — Score starts at 0

Launch the game.

You should see:

</> Plain text

Lives: 3

Score: 0

Test 2 — Reach the goal

Use the arrow keys to move the frog upward until it reaches the green goal row at the top.

Expected:

</> Plain text

YOU WIN! Score: 100

The frog should remain in the goal area rather than being reset.

Test 3 — Win state stops gameplay

After winning:

- Arrow keys should no longer move the frog.
 - Vehicles should stop moving.
 - The Win message should remain visible.
 - The score should remain 100 .
-

Test 4 — Vehicle collision still works

Restart with:

</> Plain text

R

The game should return to:

Plain text

Lives: 3

Score: 0

Allow a vehicle to collide with the frog.

Expected:

Plain text

Lives: 2

Score: 0

The frog should respawn at its starting position.

This confirms the Task 1 collision system and Task 2 lives system were preserved.

Test 5 — Game Over still works

Cause three vehicle collisions.

Expected sequence:

Plain text

3 → 2 → 1 → 0

At zero:

Plain text

GAME OVER - Press R to restart

should appear.

Test 6 — R restart

Press R after either Win or Game Over.

Expected:

[Plain text](#)

Lives: 3

Score: 0

The frog should return to the starting position and normal gameplay should resume.

What has not been implemented

This Task 3 version deliberately does **not** add:

- the 30-second timer;
- countdown display;
- timeout Game Over;
- timer-related score;
- any additional scoring system;
- new goal zones;
- unrelated changes to controls/rendering.

So the progression is now:

Task 1: Rectangle-based vehicle collision 

Task 2: 3 lives + respawn + Game Over + R restart 

Task 3: Goal detection + 100-point score + Win state 

Task 4: 30-second timer — **not implemented yet**.

Tasks 1, 2, and 3 have been implemented and tested successfully.

Please now work **ONLY** on Task 4 from the project README.

Task 4 — Add a 30-Second Timer

"Add a 30-second countdown for each attempt and handle timeout appropriately."

Requirements:

- Each attempt should begin with a 30-second countdown.
- The remaining time should be displayed clearly during gameplay.
- The timer should decrease continuously while the attempt is active.
- When the timer reaches zero, the current attempt should end appropriately.
- A timeout should consume one life and cause the frog to respawn if lives remain, consistent with the existing lives/respawn

system.

- If the timeout results in the final life being lost, the game should enter the Game Over state.
- Reaching the goal before the timer expires should result in the Win state.
- Pressing R should restart the game with 3 lives and a fresh 30-second timer.
- The functionality from Tasks 1, 2, and 3 must remain intact.
- Existing controls and unrelated functionality should not be changed.

Please inspect the current implementation before making changes.

For this task:

1. Explain the timer implementation and how it integrates with the existing game loop.
2. Explain how timeout interacts with the existing lives and respawn system.
3. Identify the exact file(s) that need to be modified.
4. Provide the minimal code changes required.
5. Preserve all functionality from Tasks 1–3.
6. Do not rewrite unrelated code.
7. Provide the complete updated contents of any modified file.
8. Provide a final testing procedure covering:
 - Vehicle collision detection
 - Three-life system
 - Frog respawn
 - Game Over
 - Goal detection
 - Score display
 - Win state
 - 30-second countdown
 - Timeout behaviour
 - R restart functionality

Worked for 12s

I inspected the actual uploaded project before working on Task 4. The ZIP itself is still the original starter source, so it does not contain your locally completed Task 1–3 changes. I'm therefore preserving the Task 1–3 implementation we established and adding **only the timer functionality** on top of it.

1. Timer implementation

The timer will belong to `GameEngine`, because the engine already controls:

- the frog;
- vehicles;
- collisions;
- lives;
- respawning;
- goal detection;
- Win/Game Over states.

Each active attempt gets:

`</>` Python

```
ATTEMPT_TIME = 30.0
```

The engine records when the current attempt began using:

`</>` Python

```
pygame.time.get_ticks()
```

Every frame, the remaining time is calculated from the current time rather than subtracting a fixed amount per frame. This is important because it makes the timer independent of the game's frame rate.

Conceptually:

`</>` Plain text

```
attempt starts
  ↓
30 seconds
  ↓
29...28...27...
  ↓
0
  ↓
lose one life
  ↓
respawn + fresh 30 seconds
```

Important detail

The timer is **per attempt**, not per entire game.

So if the player has:

</> Plain text

Lives: 3

Time: 0

the timeout causes:

</> Plain text

Lives: 2

Time: 30

and the frog respawns.

The same happens again for the second life.

If the third attempt times out:

</> Plain text

Lives: 0

GAME OVER

2. Timeout and the lives system

Timeout uses the same life-loss mechanism as vehicle collision.

That means there is only one place responsible for losing a life:

</> Python

```
_lose_attempt()
```

This prevents accidentally losing two lives from the same event.

For example, if a vehicle collision occurs at the same moment the timer expires, the collision is processed first and the attempt ends. The timer is then reset along with the respawn, so the player does **not** lose two lives in one frame.

3. Goal vs timer

The order is important.

The game checks:

1. vehicle collision;
2. whether the timer has expired;
3. whether the frog has reached the goal.

Therefore, the frog must reach the goal **while the timer is still above zero**.

If the player reaches the goal with time remaining:

</> Plain text

```
YOU WIN! Score: 100
```

If the 30 seconds have already expired, the attempt times out instead.

Once Win or Game Over occurs, the timer stops.

4. Files modified

Only these two files need to be modified:

</> Plain text

```
frogger/game/game_engine.py  
frogger/game/renderer.py
```

collisions.py remains unchanged from Task 1.

frog.py , vehicle.py , and main.py remain unchanged.

5. Complete updated game_engine.py

This version preserves Tasks 1–3 and adds Task 4.

</> Python

```
"""
```

```
GameEngine: owns the frog and all vehicles, and runs one frame's worth
```

of game logic.

Implemented functionality:

- Task 1: vehicle collision detection
- Task 2: 3-life system, respawn, Game Over, R restart
- Task 3: goal detection, score tracking, Win state
- Task 4: 30-second timer for each attempt

The timer is intentionally implemented without changing the existing controls or game structure.

"""

import random

import math

import pygame

from game.frog import Frog

6. Complete updated `renderer.py`

There is no need for a new timer-specific drawing function. The existing `draw_text()` is sufficient.

`</>` Python

"""

renderer: all pygame drawing lives here, kept separate from game logic.

"""

import pygame

CELL_SIZE = 50

GRID_COLS = 12

GRID_ROWS = 8

WIDTH = CELL_SIZE * GRID_COLS

HEIGHT = CELL_SIZE * GRID_ROWS

WINDOW_SIZE = (WIDTH, HEIGHT)

GOAL_ROW = 0

ROAD_ROWS = list(range(1, GRID_ROWS - 1))

START_ROW = GRID_ROWS - 1

COLOR_BG = (20, 20, 25)

COLOR_GOAL = (40, 130, 60)

7. Testing procedure

Run:

`</> Bash`

```
python main.py
```

1. Vehicle collision detection

Move onto a road and allow a vehicle to overlap the frog.

Expected:

`</> Plain text`

```
Lives: 2
```

```
Score: 0
```

```
Time: 30s
```

The frog should respawn.

This confirms Task 1 still works.

2. Three-life system

Cause three vehicle collisions.

Expected:

`</> Plain text`

```
3 lives → 2 lives → 1 life → 0 lives
```

After the third collision:

`</> Plain text`

```
GAME OVER - Press R to restart
```

3. Frog respawn

After the first or second collision:

- frog returns to its starting position;

- lives decrease by exactly one;
- timer returns to approximately 30 seconds.

For example:

</> Plain text

Before collision:

Lives: 3

Time: 17s

After collision:

Lives: 2

Time: 30s

4. Game Over

Lose the final life through collision.

Expected:

</> Plain text

Lives: 0

GAME OVER - Press R to restart

The timer should stop.

5. Goal detection

Restart with R and move the frog to the green goal zone before the timer expires.

Expected:

</> Plain text

Score: 100

YOU WIN! Score: 100

The timer should stop once Win is reached.

6. Score display

At the beginning:

</> Plain text

Score: 0

After reaching the goal:

</> Plain text

Score: 100

Restarting with R must return it to:

</> Plain text

Score: 0

7. Win state

After reaching the goal:

- frog remains at the goal;
- vehicles stop moving;
- arrow keys no longer move the frog;
- timer stops;
- Win message remains visible.

8. 30-second countdown

Start a fresh game and watch:

</> Plain text

Time: 30s

Time: 29s

Time: 28s

...

Time: 1s

Time: 0s

The countdown should take approximately **30 real seconds**, independent of the game's frame rate.

9. Timeout behaviour

Do not reach the goal and avoid vehicles until the timer expires.

Expected after the first timeout:

</> Plain text

Lives: 2

Time: 30s

The frog should respawn.

Allow the next attempt to expire:

</> Plain text

Lives: 1

Time: 30s

Allow the third attempt to expire:

</> Plain text

Lives: 0

GAME OVER - Press R to restart

Thus timeout behaves exactly like losing a life through collision.

10. R restart

Press R during:

- normal gameplay;
- after a timeout;
- after Win;
- after Game Over.

Every time, the game should return to:

</> Plain text

Lives: 3

Score: 0

Time: 30s

with the frog at its starting position.

Final Task 4 status

The progression is now:

Task	Functionality
Task 1	Vehicle collision detection 
Task 2	3 lives + respawn + Game Over + R restart 
Task 3	Goal + score + Win state 
Task 4	30-second per-attempt timer + timeout life loss 

Task 4 does not introduce or modify any Task 5 functionality, and no unrelated controls or game behaviour are changed.